

Asyncio Is All Your Agents Need — attendee handout

Scaling multi-model AI pipelines with the standard library · Python 3.11+ · zero dependencies

Repo: `<github.com/.../asyncio-agents-workshop>` · `python bench/run_benchmark.py --chart` · `TIME_SCALE=200 python -m pytest -q`

The one-sentence version

An agent is a loop that spends most of its life waiting on sockets. Every production problem that shows up when it leaves the notebook has a name, every name maps to one asyncio primitive, and each primitive owns one metric you can watch move.

problem	primitive	the metric it owns
independent tool calls awaited one after another	<code>asyncio.TaskGroup</code>	latency = slowest call, not the sum
provider rejects with 429 / quota sits idle	<code>asyncio.Semaphore</code>	429s → 0, in-flight == cap
fast planner OOMs the worker	<code>asyncio.Queue(maxsize=...)</code>	peak memory flat at any N
one hung request stalls the batch	<code>asyncio.timeout()</code> + hedging	p99 / wall time
one exception kills 500 jobs	<code>gather(return_exceptions=True)</code> + typed retry	jobs delivered
40-second spinner	<code>async def ... yield</code>	time to first result

Foundations in one page

An agent is a loop: reason → call tools → reason → act on model backends → verify → repeat until a stop condition. Roughly 1 % of that loop is CPU; the rest is waiting on sockets.

A multi-agent system is a graph of such loops passing messages. Four topologies cover most systems: *pipeline* (planner → generator → QA), *supervisor/workers* (fan out, collect), *critic/debate* (generate, judge, revise), *router/specialists* (classify, dispatch). Whatever the shape: agents pass messages (edges are queues), run at different speeds, fail independently, and mostly wait.

The lifecycle and the primitive that owns each stage:

stage	what happens	primitive
spawn	task arrives with a budget, deadline, parent scope	<code>TaskGroup</code> scope
gather	independent tool calls for context	<code>TaskGroup</code> (parallel)
plan	LLM decomposes into N sub-tasks (a stream, not a list)	async generator
act	model/tool calls — slow, capped, flaky	<code>Semaphore</code> · <code>timeout</code> · <code>hedge</code>
observe	verify each result (QA model, schema, guardrail)	<code>gather(return_exceptions=True)</code>
recover	retry transient, fall back, or escalate	typed <code>RetryPolicy</code>
emit / exit	stream out; release every resource on exit	<code>yield</code> · <code>finally</code> · <code>cancel()</code>

How model APIs behave — facts you design for, not bugs you fix:

behaviour	consequence	design
long-tailed latency (p50 4 s, p99 40 s, rare 300 s hang); output length drives it	one hung call stalls a batch	<code>asyncio.timeout</code> + hedging
three rate limits: RPM, TPM, hard concurrency caps that 429 instantly	naive fan-out is rejected	one <code>Semaphore</code> per provider
failures are typed: 429 slow down · 5xx retry · policy 4xx never · context 4xx shorten	blanket retry wastes money and hides bugs	typed <code>RetryPolicy</code>
non-deterministic output (deformed hands, invalid JSON)	every output needs a verifier	QA / critic stage
streaming; users judge by first token	batch APIs feel slow	async generators
cost per token / image; idle quota and duplicate calls both cost	utilisation is a P&L line	<code>Semaphore</code> · <code>cancel</code> losers

What `asyncio` is: one thread, one event loop, many coroutines. A coroutine pauses at `await` and is resumed when its I/O is ready; the loop runs whichever coroutine is ready and sleeps when none is. It makes no single call faster — it lets you wait on 10 000 at once. Pick by what you wait on: network → `asyncio`; a blocking library → a thread via `asyncio.to_thread`; CPU → a process pool. An agent turn is ~99 % network.

Where it runs: fashion e-commerce imagery (cost per *approved* image, catalogue time-to-live), consumer photo apps (time to first image = completion rate), support/research agents (p95 latency, tool cost per ticket), batch enrichment and LLM-as-judge evals (wall time under a TPM quota, zero re-runs). Latency is conversion, utilisation is cost, reliability is the SLA.

The six patterns

1 • Parallel tool calls — TaskGroup

```
async with asyncio.TaskGroup() as tg:
    t_search = tg.create_task(tools.web_search(q))
    t_skus   = tg.create_task(tools.db_query(sql))
    t_style  = tg.create_task(tools.vision_describe(ref))
# the block exits only when ALL children are done
ctx = build(t_search.result(), t_skus.result(), t_style.result())
```

Mental model: a TaskGroup is a **scope**. Children cannot outlive it. If one child raises, the others are cancelled and an `ExceptionGroup` is raised — catch it with `except* SomeError as eg:`. Compare `gather()`: it does not cancel siblings on failure and is easy to forget to await.

2 • Rate limits — Semaphore

```
limits = {name: asyncio.Semaphore(p.concurrency_limit) for name, p in providers.items()}

async with limits[job.provider]:
    img = await providers[job.provider].generate(...)
await tools.qa_check(img)      # different resource → outside the limit
```

One Semaphore per distinct limit. `async with` releases the slot on success, exception **and** cancellation — which is what makes hedging (Pattern 4) safe. Replaces hand-tuned `time.sleep()` throttling, which is guesswork and under-utilises the quota.

3 • Backpressure — bounded Queue

```
q = asyncio.Queue(maxsize=4)

async def producer():
    async for job in planner.plan(n, ctx):
        await q.put(job)          # blocks when 4 are waiting → planner pauses
    for _ in range(N_WORKERS):
        await q.put(None)        # sentinel

async def worker():
    while (job := await q.get()) is not None:
        ...render...
        del job                  # release the payload now

async with asyncio.TaskGroup() as tg:
    tg.create_task(producer())
    for _ in range(N_WORKERS): tg.create_task(worker())
```

Memory $\propto$ (queue size + workers), not $\propto$ batch size. Bonus: rendering starts after the first planned job, not the last — pipelining for free.

4 • Timeouts and hedging

```
async with asyncio.timeout(30):
    return await provider.generate(...)      # TimeoutError after 30 s
```

```

primary = asyncio.create_task(call(job.provider))
done, _ = await asyncio.wait({primary}, timeout=1.5 * p50)
tasks = {primary}
if not done:
    # primary is slow → hedge
    tasks.add(asyncio.create_task(call(fallback)))
try:
    while tasks:
        done, tasks = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)
        for t in done:
            if t.exception() is None:
                return t.result()
            # first success wins
        raise last_error
finally:
    for t in tasks: t.cancel()
    # cancel the loser

```

Every external call gets a timeout, no exceptions. Hedging trades a little duplicate work on the slow tail for a p99 that looks like your p50. **Only hedge idempotent calls.**

5 • Partial failure — outcomes as values + typed retry

```

outcomes = await asyncio.gather(*coros, return_exceptions=True)
ok = [o for o in outcomes if not isinstance(o, Exception)]
bad = [(i, o) for i, o in enumerate(outcomes) if isinstance(o, Exception)]

```

```

@dataclass(frozen=True)
class RetryPolicy:
    max_attempts: int = 3
    base_backoff: float = 0.5
    def should_retry(self, exc, attempt):
        if attempt >= self.max_attempts: return False
        if isinstance(exc, PermanentError): return False # 4xx: never
        return isinstance(exc, (TransientError, RateLimitError, TimeoutError))
    def backoff(self, exc, attempt):
        return self.base_backoff * 2 ** (attempt - 1) * (4 if isinstance(exc, RateLimitError) else 1)

```

Exceptions carry meaning: a 503 says *try again*, a 429 says *back off harder*, a 400 says *stop spending money*. A blanket `except Exception: retry` treats all three the same and hides real bugs. The policy is a frozen dataclass — test it without an event loop.

6 • Streaming — async generators

```

async def run_campaign(n) -> AsyncIterator[Event]:
    async for tok in planner.stream_tokens(...):
        yield Token(tok)
    ...start producer + workers in a TaskGroup, pushing into `out`...
    try:
        while (ev := await out.get()) is not None:
            yield ev
        yield Finished(ok, bad)
    finally:
        pipeline.cancel() # consumer walked away → tear down

async for ev in run_campaign(40): # CLI, websocket, UI — same three lines
    match ev:
        case Token(t): print(t, end="")
        case JobDone(img): show(img)

```

Also: `for fut in asyncio.as_completed(tasks): r = await fut` for “in completion order”.

What the running project measured (40 jobs)

stage	change	wall (sim s)	delivered	429s	peak mem	first result
v0	sequential, blocking	crashed @ job 2 (588 projected)	2/40	0	80 MB	never
v1	+ TaskGroup, unbounded gather	309	22/40	16	80 MB	309 s
v2	+ Semaphore per provider	309	38/40	0	80 MB	309 s
v3	+ bounded Queue	309	38/40	0	34 MB	309 s
v4	+ timeout + hedging	37	38/40	0	38 MB	37 s
v5	+ typed retry	41	40/40	0	36 MB	41 s
v6	+ streaming	42	40/40	0	34 MB	8 s

Hands-on (15 min): project/exercise/agent_todo.py

```
python project/exercise/agent_todo.py → ok=22/40 429s=16 wall=309 s
```

TODO	edit	lines	what moves
1	<code>limits = {name: Semaphore(p.concurrency_limit) ...}; async with limits[job.provider]:</code>	3	429s → 0, delivered → 38
2	<code>async with asyncio.timeout(sim(CALL_TIMEOUT)):</code>	2	wall 309 s → ~40 s
3	<code>for attempt in (1, 2): try ... except (TransientError, TimeoutError)</code>	~6	delivered → 40/40

Target: ok=40/40 429s=0 wall < 60 s. Solution in `agent_solution.py`. Done early? Add the bounded queue from `v3_queue.py` and watch `peak_mem` fall 80 → ~34 MB.

Five gotchas

1. **Blocking calls inside coroutines.** `requests.get`, `time.sleep`, a sync SDK, PIL on a 20 MB image — one blocks the whole loop. Use async clients or `await asyncio.to_thread(fn, ...)`.
`PYTHONASYNCIODEBUG=1` reports slow callbacks.
2. **Fire-and-forget `create_task()`**. No reference → the task can be garbage-collected mid-flight; if it raises, no one hears. Put it in a TaskGroup.
3. **Semaphore per process ≠ per provider.** 4 containers × `Semaphore(8)` = 32 in flight. Divide the cap by N, or centralise it (Redis token bucket).
4. **Swallowing `CancelledError`**. It is a `BaseException` on purpose. `except Exception` is fine; `except BaseException: pass` turns cancellation off. Re-raise it.
5. **Hedging non-idempotent calls.** Race two image generations, fine. Race two “charge the card” calls and you’ve charged twice.

When have you outgrown asyncio?

question	asyncio is enough	reach for an orchestrator
Where does the work run?	one process, or N identical containers	heterogeneous fleet, GPU/CPU scheduling
What happens on a crash?	re-run the batch; jobs are idempotent	must resume mid-workflow, exactly once
How long does a job live?	seconds to hours	days; must survive deploys
Who owns the infra?	the team that owns the agent	a platform team, many consumers
What breaks at 3 a.m.?	a stack trace in your code	a DAG you inspect in a UI

Scale-out for most teams: put the v6 worker in a container, feed it from a managed queue, run N copies. The async client code does not change. If none of the right-hand column applies, an orchestrator is a dependency, not a solution — and you can add one later without rewriting the worker.

Debugging tips

- Name tasks: `create_task(coro, name=f"{provider}:{job_id}")` — shows up in tracebacks and `asyncio.all_tasks()`.
- `except* SomeError as eg:` — `eg.exceptions` keeps every sibling's traceback.
- Tests need no plugin: `asyncio.run(main())` inside a plain pytest function. Scale mock time (`TIME_SCALE=200`) so the suite runs in seconds.
- `asyncio.timeout()` and `TaskGroup` are 3.11+. On 3.10 use `asyncio.wait_for()` and `gather()` with manual cancellation — and upgrade.